

LAPORAN PRAKTIKUM STRUKTUR DATA

“Implementasi Sorting dan GUI pada Java”

disusun Oleh:

Az Zahrand Solichul Tajussalathin

NIM 2511532001

Dosen Pengampu: Dr. Wahyudi S.T M.T

Asisten Pratikum: Jovantri Immanuel Gulo



FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2026

DAFTAR ISI

DAFTAR ISI	i
KATA PENGANTAR.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum	1
BAB II PEMBAHASAN	2
2.1 Program InsertionSort.....	2
2.2 Program SelectionSort.....	3
2.3 Program BubbleSort.....	4
2.4 Program InsertionSortGUI	4
BAB III TUGAS	9
3.1 Program Mahasiswa	9
3.2 Program TugasSortingGUI	10
BAB IV KESIMPULAN	15
4.1 Ringkasan Hasil Praktikum	15
DAFTAR PUSTAKA.....	iii

KATA PENGANTAR

Puji syukur kehadiran Allah SWT atas limpahan rahmat dan karunia-Nya sehingga laporan praktikum dengan judul “Implementasi Sorting dan GUI pada Java” ini dapat disusun dan diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pelaksanaan kegiatan praktikum pada mata kuliah Struktur Data.

Ucapan terima kasih disampaikan kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu, serta kepada asisten laboratorium yang telah memberikan bimbingan selama kegiatan praktikum berlangsung.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi penulisan maupun isi. Oleh karena itu, saran dan kritik yang bersifat membangun sangat diharapkan demi perbaikan di masa yang akan datang. Harapannya, laporan ini dapat memberikan manfaat dan menjadi referensi bagi pembelajaran dalam bahasa pemrograman Java.

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Pengurutan data (Sorting) merupakan operasi fundamental. Data yang tidak terurut akan sangat sulit dan tidak efisien untuk dicari atau dianalisis. Terdapat berbagai macam algoritma pengurutan yang dikembangkan, mulai dari yang paling sederhana hingga yang paling kompleks, beberapa algoritma dasarnya antara lain seperti Insertion Sort, Selection Sort, dan Bubble Sort.

1.2 Tujuan Praktikum

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar dan perbedaan logika kerja dari algoritma sorting Insertion Sort, Selection Sort, dan Bubble Sort.
2. Mengimplementasikan ketiga algoritma tersebut ke dalam Java berbasis array.
3. Membuat GUI sederhana menggunakan library javax.swing untuk memvisualisasikan proses eksekusi algoritma Insertion Sort

1.3 Manfaat Praktikum

Manfaat yang diperoleh dari praktikum ini antara lain:

1. Mengasah kemampuan analisis komparatif dalam memilih algoritma pengurutan yang tepat sesuai dengan kondisi data.
2. Melatih kemampuan dalam mengintegrasikan logika algoritma ke dalam GUI

BAB 2

PEMBAHASAN

2.1 Program InsertionSort

```
package pekan7_2511532001;

public class InsertionSort_2511532001 {
    public static void InsertionSort_2511532001_(int[] arr){
        int n_2001 = arr.length;
        for (int i = 1; i < n_2001; i++) {
            int key_2001 = arr[i];
            int j_2001 = i-1;

            while (j_2001 >= 0 && arr[j_2001] > key_2001) {
                arr[j_2001+1] = arr[j_2001];
                j_2001--;
            }
            arr[j_2001+1] = key_2001;
        }
    }

    public static void main(String[] args) {
        int arr[] = { 23,78,45,8,32,56,1 };
        int n_2001 = arr.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i = 0; i < n_2001; i++)
            System.out.print(arr[i] + " ");
        System.out.println(" ");
        InsertionSort_2511532001(arr);
        System.out.printf("array yang terurut:\n");
        for (int i = 1; i < n_2001; i++)
            System.out.print(arr[i] + " ");
        System.out.println(" ");
    }
}
```

Program InsertionSort mengimplementasikan algoritma pengurutan. Algoritma ini membagi array menjadi dua bagian: bagian yang sudah terurut dan yang belum terurut. Program menggunakan perulangan for untuk mengambil satu elemen (key) dari bagian yang belum terurut, kemudian menggunakan perulangan while untuk menggeser elemen-elemen yang lebih besar dari key ke kanan. Setelah menemukan

posisi yang tepat, key akan disisipkan. Proses ini terus berulang hingga seluruh elemen array (23, 78, 45, 8, 32, 56, 1) berada pada posisi yang terurut.

2.2 Program SelectionSort

```
package pekan7_2511532001;

public class SelectionSort_2511532001 {
    public static void selectionSort_2001(int[] arr_2001) {
        int n_2001 = arr_2001.length;
        for (int i_2001 = 0; i_2001 < n_2001; i_2001++) {
            int minIndex_2001 = i_2001;
            for (int j_2001 = i_2001 + 1; j_2001 < n_2001; j_2001++) {
                if (arr_2001[j_2001] < arr_2001[minIndex_2001]) {
                    minIndex_2001 = j_2001;
                }
            }
            int temp_2001 = arr_2001[i_2001];
            arr_2001[i_2001] = arr_2001[minIndex_2001];
            arr_2001[minIndex_2001] = temp_2001;
        }
    }

    public static void main(String[] args) {
        int arr_2001[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_2001 = arr_2001.length;
        System.out.print("array yang belum terurut:\n");
        for (int i_2001 = 0; i_2001 < n_2001; i_2001++)
            System.out.print(arr_2001[i_2001] + " ");
        System.out.println("");
        selectionSort_2001(arr_2001);
        System.out.print("array yang terurut:\n");
        for (int i_2001 = 0; i_2001 < n_2001; i_2001++)
            System.out.print(arr_2001[i_2001] + " ");
        System.out.println("");
    }
}
```

Berbeda dengan Insertion, program SelectionSort bekerja dengan cara mencari elemen bernilai paling kecil (minimum) dari seluruh sisa array yang belum terurut, lalu menukarnya (swapping) dengan elemen di posisi terdepan. Program ini bergantung pada struktur nested loop. Perulangan luar berfungsi untuk menentukan batas indeks awal (i), sementara perulangan dalam (j) bertugas memindai nilai terkecil untuk disimpan indeksinya ke dalam variabel minIndex. Setelah pemindaian selesai, pertukaran data dilakukan menggunakan variabel bantuan temp.

2.3 Program BubbleSort

```
package pekan7_2511532001;

public class BubbleSort_2511532001 {
    public static void bubbleSort_2001(int[] arr_2001) {
        int n_2001 = arr_2001.length;
        for (int i_2001 = 0; i_2001 < n_2001; i_2001++) {
            for (int j_2001 = 0; j_2001 < n_2001 - i_2001 - 1; j_2001++) {
                if (arr_2001[j_2001] > arr_2001[j_2001 + 1]) {
                    int temp_2001 = arr_2001[j_2001];
                    arr_2001[j_2001] = arr_2001[j_2001 + 1];
                    arr_2001[j_2001 + 1] = temp_2001;
                    // System.out.println("data:"+arr[j]+" "+arr[j+1]);
                }
            }
        }
    }

    public static void main(String[] args) {
        int arr_2001[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_2001 = arr_2001.length;
        System.out.print("array yang belum terurut:");
        for (int i_2001 = 0; i_2001 < n_2001; i_2001++)
            System.out.print(arr_2001[i_2001] + " ");
        System.out.println("");
        bubbleSort_2001(arr_2001);
        System.out.print("array yang terurut menggunakan BubleSort:");
        for (int i_2001 = 0; i_2001 < n_2001; i_2001++)
            System.out.print(arr_2001[i_2001] + " ");
        System.out.println("");
    }
}
```

Program BubbleSort merupakan algoritma pengurutan yang paling sederhana namun beban komputasinya paling tinggi, karena algoritma ini terus-menerus membandingkan dua elemen yang posisinya bersebelahan ($arr[j]$ dan $arr[j + 1]$). Jika elemen kiri lebih besar dari elemen kanan, program akan langsung menukar posisi keduanya, nilai terbesar ke ujung paling kanan array pada setiap iterasinya, hingga akhirnya tidak ada lagi elemen yang perlu ditukar.

2.4 Program InsertionSortGUI

```
package pekan7_2511532001;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
```

```

import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.JTextArea;
import javax.swing.border.EmptyBorder;

public class InsertionSortGUI_2511532001 extends JFrame {

    private static final long serialVersionUID_2001 = 1L;
    private int[] array_2001;
    private JLabel[] labelArray_2001;
    private JButton stepButton_2001, resetButton_2001, setButton_2001;
    private JTextField inputField_2001;
    private JPanel panelArray_2001;
    private JTextArea stepArea_2001;

    private int i_2001 = 1, j_2001;
    private boolean sorting_2001 = false;
    private int stepCount_2001 = 1;

    private JPanel contentPane_2001;

    /**
     * Create the frame.
     */
    public InsertionSortGUI_2511532001() {
        setTitle("Insertion Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_2001 = new JPanel(new FlowLayout());
        inputField_2001 = new JTextField(30);
        setButton_2001 = new JButton("Set Array");
        inputPanel_2001.add(new JLabel("Masukkan angka (pisahkan dengan
koma):"));
        inputPanel_2001.add(inputField_2001);
        inputPanel_2001.add(setButton_2001);

        // Panel array visual

```



```

        panelArray_2001 = new JPanel();
        panelArray_2001.setLayout(new FlowLayout());

        // Panel kontrol
        JPanel controlPanel_2001 = new JPanel();
        stepButton_2001 = new JButton("Langkah Selanjutnya");
        resetButton_2001 = new JButton("Reset");
        stepButton_2001.setEnabled(false);
        controlPanel_2001.add(stepButton_2001);
        controlPanel_2001.add(resetButton_2001);

        // Area teks untuk log langkah-langkah
        stepArea_2001 = new JTextArea(8, 60);
        stepArea_2001.setEditable(false);
        stepArea_2001.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane_2001 = new JScrollPane(stepArea_2001);

        // Tambahkan panel ke frame
        add(inputPanel_2001, BorderLayout.NORTH);
        add(panelArray_2001, BorderLayout.CENTER);
        add(controlPanel_2001, BorderLayout.SOUTH);
        add(scrollPane_2001, BorderLayout.EAST);

        // Event Set Array
        setButton_2001.addActionListener(e_2001 ->
setArrayFromInput_2001());

        // Event Langkah Selanjutnya
        stepButton_2001.addActionListener(e_2001 -> performStep_2001());

        // Event Reset
        resetButton_2001.addActionListener(e_2001 -> reset_2001());
    }
    private void setArrayFromInput_2001() {
        String text_2001 = inputField_2001.getText().trim();
        if (text_2001.isEmpty()) return;
        String[] parts_2001 = text_2001.split(",");
        array_2001 = new int[parts_2001.length];
        try {
            for (int k_2001 = 0; k_2001 < parts_2001.length; k_2001++) {
                array_2001[k_2001] =
Integer.parseInt(parts_2001[k_2001].trim());
            }
        } catch (NumberFormatException e_2001) {
            JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang
dipisahkan "
                + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
    }

    i_2001 = 1;
    stepCount_2001 = 1;
    sorting_2001 = true;

```

```

        stepButton_2001.setEnabled(true);
        stepArea_2001.setText("");
        panelArray_2001.removeAll();
        labelArray_2001 = new JLabel[array_2001.length];

        for (int k_2001 = 0; k_2001 < array_2001.length; k_2001++) {
            labelArray_2001[k_2001] = new
JLabel(String.valueOf(array_2001[k_2001]));
            labelArray_2001[k_2001].setFont(new Font("Arial", Font.BOLD,
24));

labelArray_2001[k_2001].setBorder(BorderFactory.createLineBorder(Color.BLACK));
            labelArray_2001[k_2001].setPreferredSize(new Dimension(50,
50));

labelArray_2001[k_2001].setHorizontalAlignment(SwingConstants.CENTER);
            panelArray_2001.add(labelArray_2001[k_2001]);
        }

        panelArray_2001.revalidate();
        panelArray_2001.repaint();
    }

    private void performStep_2001() {
        if (i_2001 < array_2001.length && sorting_2001) {
            int key_2001 = array_2001[i_2001];
            j_2001 = i_2001 - 1;

            StringBuilder stepLog_2001 = new StringBuilder();
            stepLog_2001.append("Langkah ").append(stepCount_2001)
                .append(": Memasukkan ").append(key_2001).append("\n");

            while (j_2001 >= 0 && array_2001[j_2001] > key_2001) {
                array_2001[j_2001 + 1] = array_2001[j_2001];
                j_2001--;
            }
            array_2001[j_2001 + 1] = key_2001;

            updateLabels_2001();
            stepLog_2001.append("Hasil:
").append(arrayToString_2001(array_2001)).append("\n\n");
            stepArea_2001.append(stepLog_2001.toString());

            i_2001++;
            stepCount_2001++;

            if (i_2001 == array_2001.length) {
                sorting_2001 = false;
                stepButton_2001.setEnabled(false);
                JOptionPane.showMessageDialog(this, "Sorting selesai!");
            }
        }
    }
}

```

```

        private void updateLabels_2001() {
            for (int k_2001 = 0; k_2001 < array_2001.length; k_2001++) {
labelArray_2001[k_2001].setText(String.valueOf(array_2001[k_2001]));
            }
        }

        private void reset_2001() {
            inputField_2001.setText("");
            panelArray_2001.removeAll();
            panelArray_2001.revalidate();
            panelArray_2001.repaint();
            stepArea_2001.setText("");
            stepButton_2001.setEnabled(false);
            sorting_2001 = false;
            i_2001 = 1;
            stepCount_2001 = 1;
        }

        private String arrayToString_2001(int[] arr_2001) {
            StringBuilder sb_2001 = new StringBuilder();
            for (int k_2001 = 0; k_2001 < arr_2001.length; k_2001++) {
                sb_2001.append(arr_2001[k_2001]);
                if (k_2001 < arr_2001.length - 1) sb_2001.append(", ");
            }
            return sb_2001.toString();
        }

        public static void main(String[] args) {
            SwingUtilities.invokeLater(() -> {
                InsertionSortGUI_2511532001 gui_2001 = new
InsertionSortGUI_2511532001();
                gui_2001.setVisible(true);
            });
        }
    }
}

```

Program InsertionSortGUI adalah bentuk GUI dari program logika. Program ini menggunakan class JFrame dari pustaka javax.swing untuk membentuk window aplikasi. Komponen interaktif yang digunakan meliputi JTextField untuk menampung input array pengguna, JButton untuk aksi (Set Array, Step, Reset), serta JLabel yang dibingkai (BorderFactory) untuk memvisualisasikan blok-blok elemen array. Logika Insertion Sort pada program ini dimodifikasi dan dipisahkan ke dalam method performStep agar dapat dikendalikan secara manual oleh pengguna melalui klik tombol, yang kemudian riwayat perubahannya akan dicatat ke dalam JTextArea (stepArea).

BAB 3

TUGAS

3.1 Program Mahasiswa

```

4. package pekan7_2511532001;
5.
6. public class Mahasiswa_2511532001 {
7.     private String nama_2001;
8.     private String nim_2001;
9.     private String prodi_2001;
10.
11.     // constructor
12.     public Mahasiswa_2511532001(String nama_2001, String nim_2001,
String prodi_2001) {
13.         this.nama_2001 = nama_2001;
14.         this.nim_2001 = nim_2001;
15.         this.prodi_2001 = prodi_2001;
16.     }
17.
18.     // getter dan setter
19.     public String getNama_2001() { return nama_2001; }
20.     public void setNama_2001(String nama_2001) { this.nama_2001 =
nama_2001; }
21.
22.     public String getNim_2001() { return nim_2001; }
23.     public void setNim_2001(String nim_2001) { this.nim_2001 =
nim_2001; }
24.
25.     public String getProdi_2001() { return prodi_2001; }
26.     public void setProdi_2001(String prodi_2001) { this.prodi_2001 =
prodi_2001; }
27.
28.     // toString untuk menampilkan teks di GUI
29.     @Override
30.     public String toString() {
31.         return nama_2001 + " (" + nim_2001 + " - " + prodi_2001 +
")";
32.     }
33. }

```

Penyimpanan data diimplementasikan menggunakan class ADT bernama Mahasiswa. Class ini berperan sebagai entitas objek tunggal yang membungkus tiga atribut berjenis String, yaitu nama, nim, dan prodi. Class ini dilengkapi dengan constructor untuk memfasilitasi inisialisasi data, getter-setter untuk memanipulasi akses atribut, serta method overriding toString() untuk mengonversi objek menjadi format teks (String).

3.1 Program TugasSortingGUI

```
package pekan7_2511532001;

import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.ArrayList;

public class TugasSortingGUI_2511532001 extends JFrame {
    private ArrayList<Mahasiswa_2511532001> listMahasiswa_2001;

    private JTextField txtNama_2001, txtNim_2001, txtProdi_2001;
    private JComboBox<String> comboSort_2001;
    private JTextArea txtVisualisasi_2001;

    public TugasSortingGUI_2511532001() {
        listMahasiswa_2001 = new ArrayList<>();

        setTitle("Program Sorting Mahasiswa - 2511532001");
        setSize(600, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // panel Input (atas)
        JPanel panelInput_2001 = new JPanel(new GridLayout(4, 2, 5, 5));
        panelInput_2001.setBorder(BorderFactory.createEmptyBorder(10, 10,
10, 10));

        panelInput_2001.add(new JLabel("Nama Mahasiswa:"));
        txtNama_2001 = new JTextField();
        panelInput_2001.add(txtNama_2001);

        panelInput_2001.add(new JLabel("NIM:"));
        txtNim_2001 = new JTextField();
        panelInput_2001.add(txtNim_2001);

        panelInput_2001.add(new JLabel("Program Studi:"));
        txtProdi_2001 = new JTextField();
        panelInput_2001.add(txtProdi_2001);

        JButton btnTambah_2001 = new JButton("Tambah Data");
        JButton btnReset_2001 = new JButton("Reset Data");
        panelInput_2001.add(btnTambah_2001);
        panelInput_2001.add(btnReset_2001);

        add(panelInput_2001, BorderLayout.NORTH);

        // panel tengah (visualisasi dan tabel)
        txtVisualisasi_2001 = new JTextArea();
    }
}
```

```

        txtVisualisasi_2001.setEditable(false);
        txtVisualisasi_2001.setFont(new Font("Monospaced", Font.PLAIN,
12));
        JScrollPane scroll_2001 = new JScrollPane(txtVisualisasi_2001);
        scroll_2001.setBorder(BorderFactory.createTitledBorder("Visualisasi
Sorting & Hasil"));
        add(scroll_2001, BorderLayout.CENTER);

        // panel bawah (kontrol sorting)
        JPanel panelKontrol_2001 = new JPanel(new FlowLayout());
        String[] metodeSort_2001 = {"Insertion Sort", "Selection Sort",
"Bubble Sort"};
        comboSort_2001 = new JComboBox<>(metodeSort_2001);
        JButton btnSort_2001 = new JButton("Mulai Sorting");

        panelKontrol_2001.add(new JLabel("Pilih Metode: "));
        panelKontrol_2001.add(comboSort_2001);
        panelKontrol_2001.add(btnSort_2001);

        add(panelKontrol_2001, BorderLayout.SOUTH);

        // event listener tombol tambah
        btnTambah_2001.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                String nama_2001 = txtNama_2001.getText();
                String nim_2001 = txtNim_2001.getText();
                String prodi_2001 = txtProdi_2001.getText();

                if (!nama_2001.isEmpty() && !nim_2001.isEmpty() &&
!prodi_2001.isEmpty()) {
                    listMahasiswa_2001.add(new
Mahasiswa_2511532001(nama_2001, nim_2001, prodi_2001));
                    tampilkanDataMentah_2001();
                    txtNama_2001.setText(""); txtNim_2001.setText("");
                    txtProdi_2001.setText("");
                } else {
                    JOptionPane.showMessageDialog(null, "Isi semua data!");
                }
            }
        });

        // event listener tombol rset
        btnReset_2001.addActionListener(e -> {
            listMahasiswa_2001.clear();
            txtVisualisasi_2001.setText("Data telah di-reset.");
        });

        // event listener tombol sorting
        btnSort_2001.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (listMahasiswa_2001.isEmpty()) {

```

```

        JOptionPane.showMessageDialog(null, "Data kosong!
Tambahkan data dulu.");
        return;
    }

    // copy data agar array list asli tidak berubah (untuk
    mengetes algo lain)
    ArrayList<Mahasiswa_2511532001> listCopy_2001 = new
    ArrayList<>(listMahasiswa_2001);
    String pilihan_2001 = (String)
    comboSort_2001.getSelectedItem();

    txtVisualisasi_2001.setText("== " +
    pilihan_2001.toUpperCase() + " ==\n");
    txtVisualisasi_2001.append("Data Awal: " +
    dapatkanHanyaNama_2001(listCopy_2001) + "\n\n");

    if (pilihan_2001.equals("Insertion Sort")) {
        insertionSort_2001(listCopy_2001);
    } else if (pilihan_2001.equals("Selection Sort")) {
        selectionSort_2001(listCopy_2001);
    } else {
        bubbleSort_2001(listCopy_2001);
    }

    txtVisualisasi_2001.append("\n== HASIL AKHIR ==\n");
    for (Mahasiswa_2511532001 mhs_2001 : listCopy_2001) {
        txtVisualisasi_2001.append(mhs_2001.toString() + "\n");
    }
    }
    });
}

private void tampilkanDataMentah_2001() {
    txtVisualisasi_2001.setText("Data Saat Ini:\n");
    for (Mahasiswa_2511532001 mhs_2001 : listMahasiswa_2001) {
        txtVisualisasi_2001.append(mhs_2001.toString() + "\n");
    }
}

private String dapatkanHanyaNama_2001(ArrayList<Mahasiswa_2511532001>
list_2001) {
    StringBuilder sb_2001 = new StringBuilder("[");
    for (int i = 0; i < list_2001.size(); i++) {
        sb_2001.append(list_2001.get(i).getNama_2001());
        if (i < list_2001.size() - 1) sb_2001.append(", ");
    }
    sb_2001.append("]");
    return sb_2001.toString();
}

// algoritma sorting

```

```

        private void insertionSort_2001(ArrayList<Mahasiswa_2511532001>
arr_2001) {
            int n_2001 = arr_2001.size();
            for (int i_2001 = 1; i_2001 < n_2001; i_2001++) {
                Mahasiswa_2511532001 key_2001 = arr_2001.get(i_2001);
                int j_2001 = i_2001 - 1;

                // compareToIgnoreCase menghasilkan > 0 jika string pertama
lebih besar secara alfabetis
                while (j_2001 >= 0 &&
arr_2001.get(j_2001).getNama_2001().compareToIgnoreCase(key_2001.getNama_20
01()) > 0) {
                    arr_2001.set(j_2001 + 1, arr_2001.get(j_2001));
                    j_2001--;
                }
                arr_2001.set(j_2001 + 1, key_2001);

                txtVisualisasi_2001.append("Langkah " + i_2001 + ": " +
dapatkanHanyaNama_2001(arr_2001) + "\n");
            }
        }

        private void selectionSort_2001(ArrayList<Mahasiswa_2511532001>
arr_2001) {
            int n_2001 = arr_2001.size();
            for (int i_2001 = 0; i_2001 < n_2001 - 1; i_2001++) {
                int minIdx_2001 = i_2001;
                for (int j_2001 = i_2001 + 1; j_2001 < n_2001; j_2001++) {
                    if
(arr_2001.get(j_2001).getNama_2001().compareToIgnoreCase(arr_2001.get(minId
x_2001).getNama_2001()) < 0) {
                        minIdx_2001 = j_2001;
                    }
                }
                // swap
                Mahasiswa_2511532001 temp_2001 = arr_2001.get(minIdx_2001);
                arr_2001.set(minIdx_2001, arr_2001.get(i_2001));
                arr_2001.set(i_2001, temp_2001);

                txtVisualisasi_2001.append("Pass " + (i_2001 + 1) + ": " +
dapatkanHanyaNama_2001(arr_2001) + "\n");
            }
        }

        private void bubbleSort_2001(ArrayList<Mahasiswa_2511532001> arr_2001)
{
            int n_2001 = arr_2001.size();
            for (int i_2001 = 0; i_2001 < n_2001 - 1; i_2001++) {
                for (int j_2001 = 0; j_2001 < n_2001 - i_2001 - 1; j_2001++) {
                    if
(arr_2001.get(j_2001).getNama_2001().compareToIgnoreCase(arr_2001.get(j_200
1 + 1).getNama_2001()) > 0) {
                        // Swap

```



```

        Mahasiswa_2511532001 temp_2001 = arr_2001.get(j_2001);
        arr_2001.set(j_2001, arr_2001.get(j_2001 + 1));
        arr_2001.set(j_2001 + 1, temp_2001);
    }
}
txtVisualisasi_2001.append("Pass " + (i_2001 + 1) + ": " +
dapatkanHanyaNama_2001(arr_2001) + "\n");
}
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new TugasSortingGUI_2511532001().setVisible(true);
    });
}
}

```

Program utama TugasSortingGUI mengintegrasikan Java Swing dengan struktur ArrayList untuk mengelola objek mahasiswa. Proses pengurutan menggunakan metode Insertion, Selection, dan Bubble Sort yang dimodifikasi khusus untuk data teks (String) dengan memanfaatkan fungsi compareToIgnoreCase().

Fungsi ini bekerja dengan membandingkan nilai ASCII tanpa membedakan huruf kapital, di mana nilai return > 0 akan memicu proses pergeseran atau swapping antar elemen. Sebelum mengeksekusi algoritma, program secara otomatis melakukan cloning data pada ArrayList agar data asli tetap utuh dan pengguna dapat menguji algoritma lain secara bergantian tanpa perlu input ulang.

BAB 4

KESIMPULAN

4.1 Ringkasan Hasil Praktikum

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Insertion Sort, Selection Sort, dan Bubble Sort merupakan tiga pendekatan berbeda dalam memecahkan masalah pengurutan data di dalam array. Ketiga algoritma ini mengandalkan perulangan bersarang untuk membandingkan dan memanipulasi posisi. Meskipun kurang cocok untuk dataset berskala masif, ketiganya memiliki keunikan, yaitu Bubble Sort unggul dalam kemudahan penulisan kode, Selection Sort meminimalisir jumlah pertukaran data secara fisik, sedangkan Insertion Sort memiliki efisiensi tertinggi jika data awal sudah dalam keadaan hampir terurut.

DAFTAR PUSTAKA

- [1] GeeksforGeeks, "Sorting Algorithms in Java". [Daring]. Tersedia pada:
<https://www.geeksforgeeks.org/sorting-algorithms/>. [Diakses: 19 Mei 2026].